BHV_AvdCollision and BHV_AvdColregs

Collision-avoidance and COLREGS behaviors.

Included MIT MOOS-IvP PDF sources:

- bhv_avdcol.pdf
- bhv_colregs.pdf

The AvoidCollision Behavior

Michael Benjamin, MIT Dept of Mechanical Engineering, mikerb@mit.edu

Contents

1 The AvoidCollision Behavior

The AvoidCollision behavior will produce IvP objective functions designed to avoid collisions (and near collisions) with another specified vehicle. The IvP functions produced by this behavior are defined over the domain of posssible heading and speed choices. The utility assigned to a point in this domain (a heading-speed pair) depends in part on the calculated closest point of approach (CPA) between the candidate maneuver leg, and the contact leg formed from the contact's position and trajectory. A further user-defined utility function is applied to the CPA calculation for a candidate maneuver.

1.1 Configuration Parameters

Listing 1.1: Configuration Parameters Common to All Behaviors.

`activeflag`:	A MOOS variable-value pair posted when the behavior is in the *active* state. [more].
`condition`:	Specifies a condition that must be met for the behavior to be running. [more].
`duration`:	Time in behavior will remain running before declaring completion. [more].
`duration_idle_decay`:	When true, duration clock is running even when in the *idle* state. [more].
`duration_reset`:	A variable-pair such as `MY_RESET=true`, that will trigger a duration reset. [more].
`duration_status`:	The name of a MOOS variable to which the vehicle duration status is published. [more].
`endflag`:	A MOOS variable-value pair posted when the behavior has completed. [more].

idleflag:	A MOOS variable-value pair posted when the behavior is in the *idle* state. [more].
inactiveflag:	A MOOS variable-value posted when the behavior is *not* in the *active* state. [more].
name:	The (unique) name of the behavior. [more].
nostarve:	Allows a behavior to assert a maximum staleness for a MOOS variable. [more].
perpetual:	If true allows the behavior to to run even after it has completed. [more].
post_mapping:	Re-direct behavior output normally to one MOOS variable to another instead. [more].
priority:	The priority weight of the behavior. [more].
pwt:	Same as priority.
runflag:	A MOOS variable and a value posted when a behavior has met its conditions. [more].
spawnflag:	A MOOS variable and a value posted when a behavior is spawned. [more].
spawnxflag:	A MOOS variable and a value posted when a behavior is spawned. [more].
templating:	Turns a behavior into a template for spawning behaviors dynamically. [more].
updates:	A MOOS variable from which behavior parameter updates are read dynamically. [more].

Listing 1.2: Configuration Parameters Common to Contact Behaviors.

Parameter	Description
bearing_lines	If true, a visual artifict will be produced for rendering the bearing line between ownship and the contact when the behavior is running. Not all behaviors implement this feature.
contact	Name or unique identifier of a contact to be avoided.
decay	Time interval during which extrapolated position slows to a halt.
exit_on_filter_group	If true, and if an exclusion filter is implemented for this contact behavior, an early exit of the behavior may be allowed when or if the group name changes and no longer satisfies the exclusion filter. The default is false.
exit_on_filter_vtype	If true, and if an exclusion filter is implemented for this contact behavior, an early exit of the behavior may be allowed when or if the vehicle type changes and no longer satisfies the exclusion filter. The default is false.
exit_on_filter_region	It true, and if an exclusion filter is implemented for this contact behavior, an early exit of the behavior may be allowed when or if the contact moves into a region that would no longer satisfy the exclusion filter. The default it false.
extrapolate	If true, contact position is extrapolated from last position and trajectory.

ignore_group:	If specified, the contact group may not match the given ignore group. If multiple ignore groups are specified, the contact group must be different than all ignore groups. Introduced after Release 19.8.1. Section **??**
ignore_name:	If specified, the contact name may not match the given ignore name. If multiple ignore names are specified, the contact name must be different than all ignore names. Introduced after Release 19.8.1. Section **??**
ignore_region:	If specified, the contact group may be in the given ignore region. If multiple ignore regions are specified, the contact position must be external to all ignore regions. Introduced after Release 19.8.1. Section **??**
ignore_type:	If specified, the contact type may not match the given ignore type. If multiple ignore types are specified, the contact type must be different than all ignore types. Introduced after Release 19.8.1. Section **??**
match_group:	If specified, the contact group must match the given match group. If multiple match groups are specified, the contact group must match at least one match group. Introduced after Release 19.8.1. Section **??**
match_name:	If specified, the contact name must match the given match name. If multiple match names are specified, the contact name must match at least one. Introduced after Release 19.8.1. Section **??**
match_region:	If specified, the contact must reside in the given convex region. If multiple match regions are specified, the contact position must be in at least one match region. The multiple regions essentially can together support a non-convex regins. Introduced after Release 19.8.1. Section **??**
match_type:	If specified, the contact type must match the given match type. If multiple match types are specified, the contact type must match at least one match type. Introduced after Release 19.8.1. Section **??**
on_no_contact_ok	If false, a helm error is posted if no contact information exists. Applicable in the more rare case that a contact behavior is statically configured for a named contact. The default is true.
strict_ignore	If true, and if one of the ignore exclusion filter components is enabled, then an exclusion filter will fail if the contact report is missing information related to the filter. For example if the contact group information is unknown. The default is true.
time_on_leg	The time on leg, in seconds, used for calculating closest point of approach.

Listing 1.3: Configuration Parameters for the AvoidCollision Behavior.

Parameter	Description
completed_dist:	Range to contact outside of which the behavior completes and dies. The default is 500 meters.
max_util_cpa_dist:	Range to contact outside which a considered maneuver will have max utility. Section 1.6. The default is 75 meters
min_util_cpa_dist:	Range to contact within which a considered maneuver will have min utility. Section 1.6. The default is 10 meters.

no_alert_request:	If true, the behavior will not send an automatic configuration message to the contact manager at startup. Section 1.4. The default is false.
pwt_grade:	Grade of priority growth as the contact moves from the pwt_outer_dist to the pwt_inner_dist. Choices are `linear`, `quadratic`, or `quasi`. The default is `quasi`. Section 1.5.
pwt_inner_dist:	Range to contact within which the behavior has maximum priority weight. Section 1.5. The default is 50 meters.
pwt_outer_dist:	Range to contact outside which the behavior has zero priority weight. Section 1.5. The default is 200 meters.
use_refinery:	If true, the behavior will produce an optimized objective function that is faster to produce, uses a smaller memory footprint, and contributes to faster helm solution time. The default is false, simply for continuity with prior releases, but there is no downside to enabling this feature. Section 1.8.

Listing 1.4: Example Configuration Block.

```
Behavior = BHV_AvoidCollision
{
  // General Behavior Parameters
  // ---------------------------
  name         = avdcollision_                     // example
  pwt          = 200                               // example
  condition    = AVOID = true                      // example
  updates      = CONTACT_INFO                      // example
  endflag      = CONTACT_RESOLVED = $[CONTACT]     // example
  templating   = spawn                             // example


  // General Contact Behavior Parameters
  // -----------------------------------
      bearing_lines = white:0, green:0.65, yellow:0.8, red:1.0   // example

            contact = henry            // example
              decay = 15,30            // default (seconds)
        extrapolate = true             // default
   on_no_contact_ok = true             // default
        time_on_leg = 60               // default (seconds)


  // Parameters specific to this behavior
  // ------------------------------------
     completed_dist = 500              // default
  max_util_cpa_dist = 75               // default
  min_util_cpa_dist = 10               // default
   no_alert_request = false            // default
          pwt_grade = quasi            // default
    pwt_inner_dist = 50                // default
    pwt_outer_dist = 200               // default
}
```

1.2 Variables Published

The below MOOS variables will be published by the behavior during normal operation, in addition to any configured flags. A variable published by any behavior may be supressed or changed to a different variable name using the post_mapping configuration parameter described in Section **??**.

- BCM_ALERT_REQUEST: A request to the contact manager specifying the conditions for contact alerts.
- CLOSING_SPD_AVD: The current closing speed, in meters per second, to the contact.
- CONTACT_RESOLVED: Posted with contact name when the behavior completes and dies.
- RANGE_AVD: The current range, in meters, to the contact.
- VIEW_SEGLIST: A bearing line between ownship and the contact if configured for rendering.

1.3 Configuring and Using the AvoidCollision Behavior

The AvoidCollision behavior produces an objective function based on the relative positions and trajectories between the vehicle and a given contact. The objective function is based on applying a utility to the calculated closest point of approach (CPA) for a candidate maneuver. The user may configure a priority weight, but this weight is typically degraded as a function of the range to the contact. The behavior may be configured for avoidance with respect to a known contact, or it may be configured to spawn a new instance upon demand as contacts present themselves.

1.4 Automatic Requests for Contact Manager Alerts

The collision avoidance behavior is most commonly configured as a *template*, meaning instances will not be spawned until an outside event, i.e, posting to the MOOSDB, is received by the helm. This was discussed in detail in Section **??**. The spawning event for the collision avoidance behavior typically comes from the pBasicContactMgr application. This app therefore needs to be informed, by the collision avoidance behavior, of the desired conditions for generating behavior-spawning alerts. This is done automatically upon helm startup with a posting of the form:

```
BCM_ALERT_REQUEST = id=avd, onflag=CONTACT_INFO=name=$[VNAME] # contact=$[VNAME],
                    alert_range=80, cpa_range=100
```

The values for this posting are chosen as follows:

- The value from the `onflag` component is a posting to be made by the contact manager when an alert is triggered. Like all MOOS postings, the posting has a variable and value component. The variable component is `CONTACT_INFO`. The value component is

  ```
  name=$[VNAME] # contact=$[VNAME]
  ```

- When the contact manager posts the onflag, it will expand the `$[VNAME]` macros with the actual name of the contact. The variable `CONTACT_INFO` is the variable provided in the updates parameter for the behavior.

- The value from the `alert_range` component, e.g., 80 in the example above, is the value specified in the `pwt_outer_dist` parameter for the behavior. This is the range between ownship and contact beyond which the behavior assigns a priority weight of zero.
- The value from the `cpa_range` component, e.g., 100 in the example above, is the value specified in the `completed_dist` parameter for the behavior. This is the range, in meters, between ownship and contact beyond which the behavior will initiate its own completion and de-instatiation.

If some other contact manager regime is being used other than `pBasicContactMgr`, the above automatic posting is probably harmless. However, if one really wants to disable this automatic posting, it can be turned off by setting the configuration parameter `no_alert_request` to true.

Implementation note: One may wonder when or how the behavior can make this automatic posting when the behavior is configured as a template. An instance is never spawned until an alert is received, but the alert parameters are posted by the behavior, creating a bit of a chicken or the egg conundrum. The alert request is actually posted by an instance of the behavior created ever so briefly at helm startup. At startup, the helm creates instances of *all* behaviors, even templates, to ensure the configuration parameters are correct. The ultimate confirmation of behavior parameter correctness is obtained by a behavior instance itself confirming each parameter. The helm will then immediately, before its first iteration, delete any behaviors temporarily created from template behaviors. During this brief startup period, the helm will invoke the function `onHelmStart()` for all behaviors. This function is defined at the IvPBehavior superclass level just like `onRunState()`. In the case of the collision avoidance behavior, this function is implemented to make the automatic alert configuration posting to `BCM_ALERT_REQUEST`.

1.5 Specifying the Behavior Priority Weight Policy

The AvoidCollision behavior may be configured to increase its priority as it closes range to the contact. The priority weight specified in its configuration represents the *maximum* possible priority applied to the behavior, presumably in close range to the contact. The range at which this maximum priority applies is specified in the `pwt_inner_dist` parameter. Likewise, the `pwt_outer_dist` parameter specifies a range to the contact where the priority weight becomes zero, regardless of the priority weight specified in the configuration file.

So the *current priority* will always be between zero and the maximum priority set in the behavior `priority` configuration parmeter. To be more precise:

Current Priority =

- 0 if current range to contct is greater than or equal to `pwt_outer_dist`
- 100 if current range to contact is less than or equal to `pwt_inner_dist`
- otherwise ((`pwt_outer_dist` - current range) / (`pwt_outer_dist` - `pwt_inner_dist`)) * priority

This relationship is shown in Figure 1.

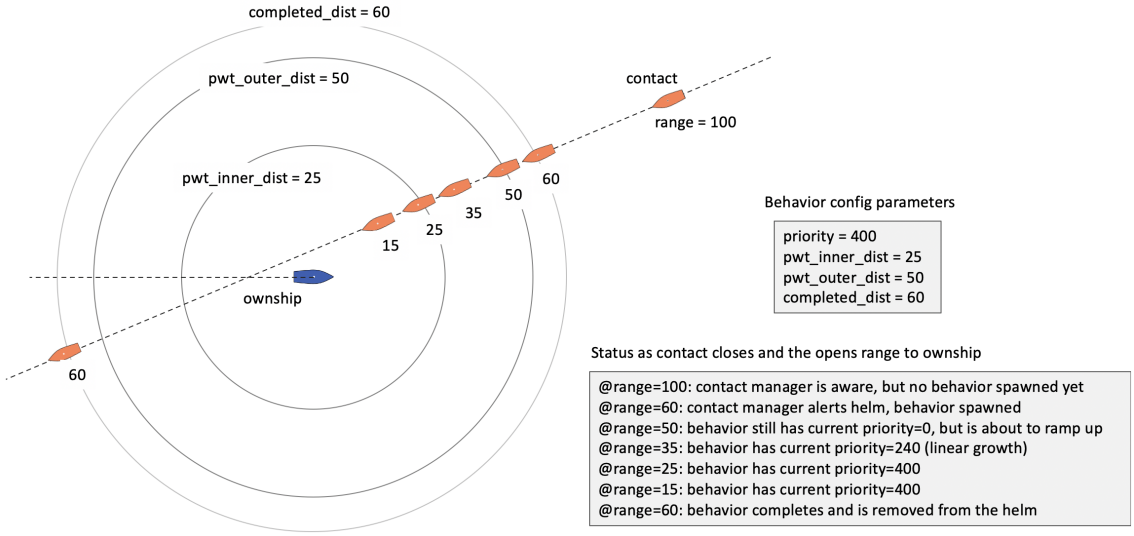

Figure 1: **Scaling priority weights based on ownship range to contact**: The range between the two vehicles affects whether the behavior is spawned, active and with what priority weight. Beyond the range specified by `completed_dist` the behavior is likely not inexistence. Beyond the range specified by `pwt_outer_dist`, the behavior is not yet active. Within the range of `pwt_outer_dist`, the behavior becomes active with a non-zero priority weight growing as the contact closes range. Within the range of `pwt_inner_dist`, the behavior is active with 100% of its configured priority weight.

The example shown below in Figure 2 shows the effect of the `pwt_outer_dist` parameter. The vehicle on the left is proceeding east, oblivious to the two approaching vessels. The two westbound vessels, `ben` and `cal` are simulated exactly on top of one another. They are oblivious to one another, but will use the collision avoidance behavior to avoid the eastbound vessel, abe. The only difference between `ben` and `cal` is that `cal` begins winding up its priority weight at 80 meters range to `abe`, as opposed to 30 meters for `ben`. The short simulation shows the resulting difference in trajectory.

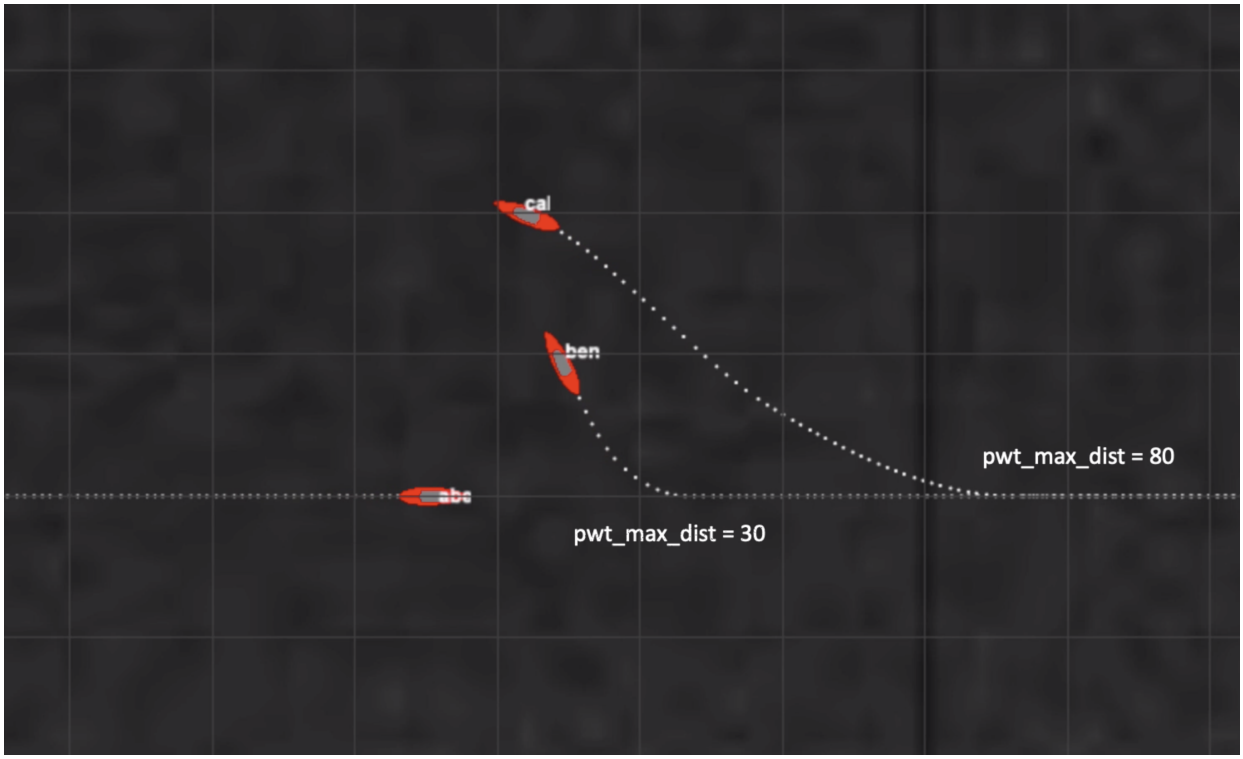

Figure 2: Two vehicles use the AvoidCollision behavior to avoid the oblivious and non-maneuvering Eastbound vehicle. One vehicle has `pwt_outer_dist` set to 80 and the other set to 30. This affects when each vehicle begins its maneuver to avoid.

`video:(0:12): https://vimeo.com/457500757`

By default, the priority weight decreases linearly between the two depicted ranges. The `pwt_grade` parameter allows the degradation from maximum priority to zero priority to fall more steeply by setting `pwt_grade=quadratric`.

1.6 Specifying the Utility Policy of the Behavior

Whereas the behavior *weight* discussed above in Section 1.5 determines the influence of the behavior relative to other behaviors, the behavior *utility function* specifies the relative utility of candidate maneuvers from the perspective of the collision avoidance goals of this behavior.

The utility function for the avoid collision behavior is based on two factors:

- The range at the closest point of approach (CPA) for a candidate maneuver
- The determination risk associated with any CPA range.

The first component is just physics. Given ownship and contact current positions and trajectories, and the assumption that the contact will stay on its current heading and speed for at least the near future, the projected CPA range may be calculated for any given heading-speed maneuver. Consider the example in Figure 3 below with particular ownship and contact positions and trajectories shown

on the left. On the right, the projected CPA range values are plotted for all candidate ownship maneuvers.

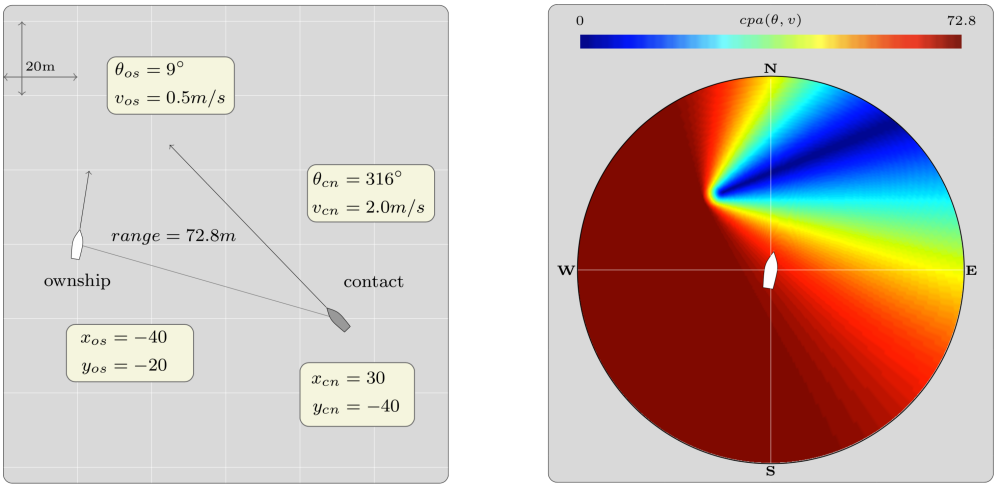

Figure 3: (left) An example collision avoidance situation, where ownship would cross behind the contact if it remained on its present heading and speed. (right) The evaluation of closest point of approach for all possible maneuvers with ownship max speed of 4 m/s. The darkest blue maneuvers will result in a collision or near collision.

The second component of the utility function is the mapping of "risk" to certain CPA range values. This part is very subjective, and of course is why there are configuration parameters, allow different users to align the behavior with their own notion of risk. The two key parameters are `min_util_cpa_dist` and `max_util_cpa_dist`. The former refers the the CPA range with the minimum utility, essentially equivalent to a collision. For this value, perhaps pick a range that, while not a collision, someone would get written up for a safety violation for breaching this range. The `max_util_cpa` range, on the other hand, is the range that, above which there is not increase in utility. Everything at this range or higher is "all clear".

These two parameters form a function, $g()$, which takes as input the CPA range value and outputs the utility of a canditate maneuver. This function is depicted on the left in Figure 4, and the overall utility function is shown on the right:

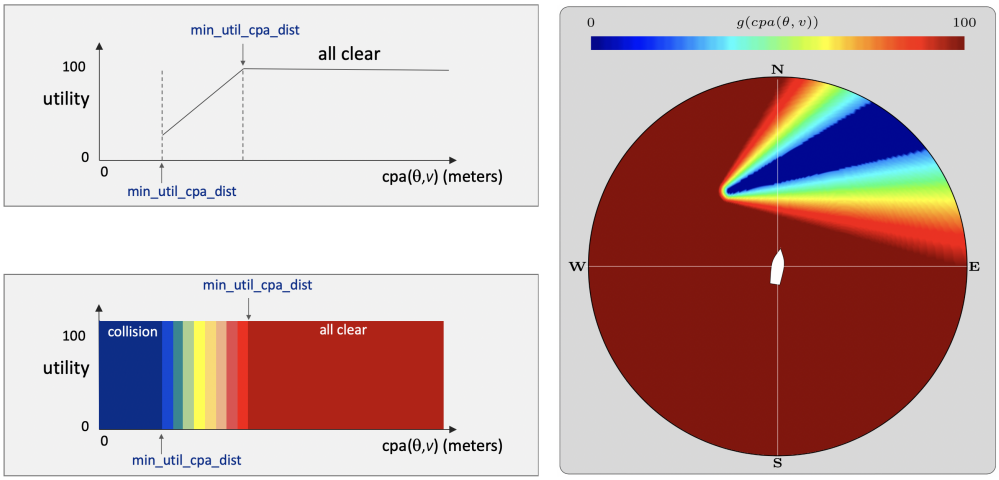

Figure 4: (left) A utility function mapping CPA range values to a single risk utility is shown. Up to the range specified by the `min_util_cpa` configuration parameter, maneuvers resulting in these ranges are considered essentially collisions. Above the range specified by the `max_util_cpa` configuration parameter, maneuvers resulting in these ranges are considered essentially equivalently optimal. In between are the ranges that are in between disaster and optimal. These represent compromise maneuvers that the helm may opt for if there are other behaviors that find such maneuvers useful for accomplishing their objectives.

The `min_util_cpa_dist` and `max_util_cpa_dist` parameters have a default value of 10 and 75 meters respectively. These values are very subjective and will need to be adjusted per vehicle and mission. Currently these defaults are used if not specified by the user, but in future releases overriding these values may be enforced.

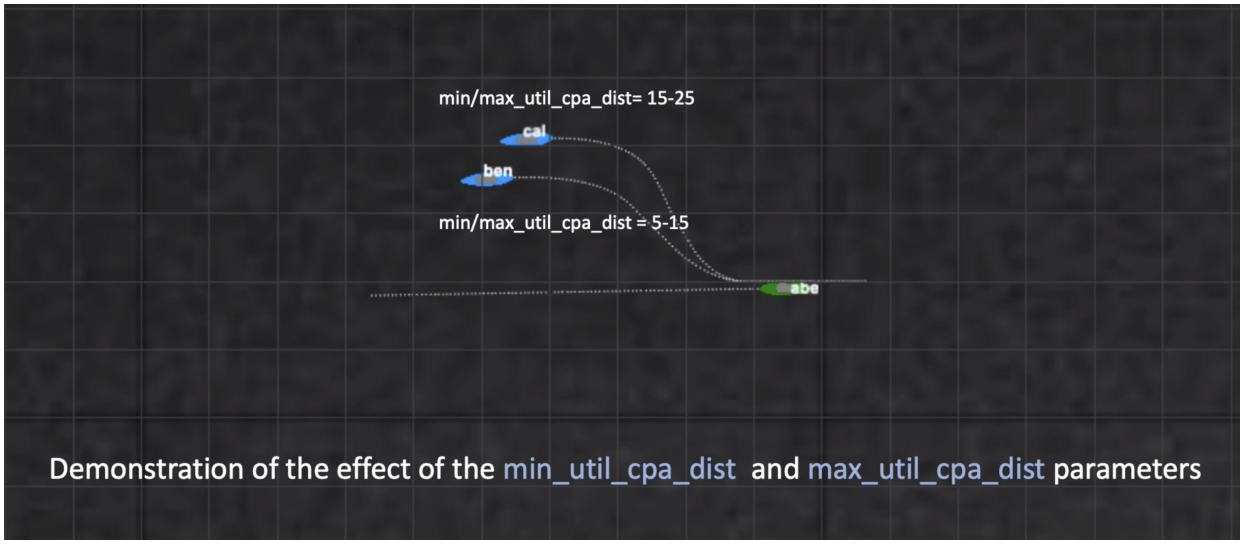

Figure 5: Two vehicles use the AvoidCollision behavior to avoid the oblivious and non-maneuvering eastbound vehicle. One vehicle, ben, is configured with the `min_util_cpa_dist` and `max_util_cpa_dist` paremeters set to 5 and 15 meters respectively. The other vehicle, cal, is more risk averse and has these to parameters set to 15 and 25 meters respectively. The resulting trajectories of ben and cal are shown. The two vehicles on the right are simulated separately, unaware of each other, with the same starting position and parameters except for two above parameters. The divergence in trajectory is soley due to the differences around these two parameters.
`video:(0:13): https://vimeo.com/458207817`

1.7 Specifying Contact Flags

Contact flags are new feature for all contact behaviors, available in releases after Release 19.8.1.

1.8 Using the CPA Refinery

1.9 Relevant Example Missions

The COLREGS Behavior

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/bhvdocs/bhv_colregs`

Contents

1 The AvdColregs Behavior

The AvdColregs behavior will produce IvP objective functions designed to avoid collisions (and near collisions) with another specified vehicle, based on the protocol found in the US Coast Guard Collision Regulations (COLREGS) citecolregs. The IvP functions produced by this behavior are defined over the domain of posssible heading and speed choices. The utility assigned to a point in this domain (a heading-speed pair) depends in part on the calculated closest point of approach (CPA) between the candidate maneuver leg, and the contact leg formed from the contact's position and trajectory. A further user-defined utility function is applied to the CPA calculation for a candidate maneuver.

Note: This behavior is invoked as `BHV_AvdColregsV17` in mission files. The `m2_berta_detect` mission is an example simulation mission for this behavior. Note finally that the configuration interface for this behavior is very similar to `BHV_AvoidCollision`, even though the effects of the behavior are quite different. Documenation is evolving for this behavior. If you would like a more detail discussion of the algorithms implementing the COLREGS behavior, please see MIT CSAIL TR-2017-09. As this remains a very active area of research in this lab, further releases and documentation are expected later in 2017 and beyond.

1.1 Configuration Parameters

Listing 1.1: Configuration Parameters for the AvdColregs Behavior.

Parameter	Description
`completed_dist`:	Range to contact outside of which the behavior completes and dies.

max_util_cpa_dist:	Range to contact outside which a considered maneuver will have max utility. Section **??**.
giveway_bow_dist:	In the giveway consideration, when determining if ok to cross the contact's bow, this range to contact at time of crossing from contact's port to starboard, must be met in order for ownship to consider crossing the contac'ts bow. This is a new option in V19 of this vehavior. If left unspecified, it will revert to the criteria used in previous versions.
min_util_cpa_dist:	Range to contact within which a considered maneuver will have min utility. Section **??**.
no_alert_request:	If true will turn off automatic handshaking with the contact manager. Section **??**.
pwt_grade:	Grade of priority growth as the contact moves from the pwt_outer_dist to the pwt_inner_dist. Choices are `linear`, `quadratic`, or `quasi`. Section **??**.
pwt_inner_dist:	Range to contact within which the behavior has maximum priority weight. Section **??**.
pwt_outer_dist:	Range to contact outside which the behavior has zero priority weight. Section **??**.
refinery:	If true, the behavior will create an IvP Function using the refinery, using large pieces for plateau regions. This is a new feature after Release 19.8, and currently only is used in the CPA or in-extremis modes. The default is false.

Listing 1.2: Example Configuration Block.

```
Behavior = BHV_AvdColregsV17
{
  // General Behavior Parameters
  // ---------------------------
  name         = avdcollision_                    // example
  pwt          = 200                              // example
  condition    = AVOID = true                     // example
  updates      = CONTACT_INFO                     // example
  endflag      = CONTACT_RESOLVED = $[CONTACT]    // example
  templating   = spawn                            // example


  // General Contact Behavior Parameters
  // -----------------------------------
      bearing_lines = white:0, green:0.65, yellow:0.8, red:1.0   // example

            contact = henry            // example
              decay = 15,30            // default (seconds)
        extrapolate = true             // default
   on_no_contact_ok = true             // default
         time_on_leg = 60              // default (seconds)


  // Parameters specific to this behavior
  // ------------------------------------
     completed_dist = 500              // default
  max_util_cpa_dist = 75               // default
  min_util_cpa_dist = 10               // default
   no_alert_request = false            // default
          pwt_grade = quasi            // default
     pwt_inner_dist = 50               // default
     pwt_outer_dist = 200              // default
}
```

1.2 Variables Published

The below MOOS variables will be published by the behavior during normal operation, in addition to any configured flags. A variable published by any behavior may be supressed or changed to a different variable name using the post_mapping configuration parameter described in Section **??**.

- BCM_ALERT_REQUEST: A request to the contact manager specifying the conditions for contact alerts.
- CLOSING_SPD_AVD: The current closing speed, in meters per second, to the contact.
- CONTACT_RESOLVED: Posted with contact name when the behavior completes and dies.
- RANGE_AVD: The current range, in meters, to the contact.
- VIEW_SEGLIST: A bearing line between ownship and the contact if configured for rendering.

1.3 Configuring and Using the AvdColregs Behavior

The AvdColregs behavior produces an objective function based on the relative positions and trajectories between the vehicle and a given contact. The objective function is based on applying a

utility to the calculated closest point of approach (CPA) for a candidate maneuver. The user may configure a priority weight, but this weight is typically degraded as a function of the range to the contact. The behavior may be configured for avoidance with respect to a known contact, or it may be configured to spawn a new instance upon demand as contacts present themselves.

1.4 Automatic Requests for Contact Manager Alerts

The collision avoidance behavior is most commonly configured as a *template*, meaning instances will not be spawned until an outside event, i.e, posting to the MOOSDB, is received by the helm. This was discussed in detail in Section **??**. The spawning event for the collision avoidance behavior typically comes from the pBasicContactMgr application. This app therefore needs to be informed, by the collision avoidance behavior, of the desired conditions for generating behavior-spawning alerts. This is done automatically upon helm startup with a posting of the form:

```
BCM_ALERT_REQUEST = id=avd, var=CONTACT_INFO, val="name=$[VNAME] # contact=$[VNAME]",
                    alert_range=80, cpa_range=100
```

The values for this posting are chosen as follows:

- The value from the `var` component, e.g., `CONTACT_INFO` in the example above, is the variable named in the `updates` parameter for the behavior.
- The value from the `alert_range` component, e.g., 80 in the example above, is the value specified in the `pwt_outer_dist` parameter for the behavior. This is the range between ownship and contact beyond which the behavior assigns a priority weight of zero.
- The value from the `alert_range_cpa` component, e.g., 100 in the example above, is the value specified in the `completed_dist` parameter for the behavior. This is the range between ownship and contact beyond which the behavior will initiate its own completion and de-instatiation.

If some other contact manager regime is being used other than pBasicContactMgr, the above automatic posting is probably harmless. However, if one really wants to disable this automatic posting, it can be turned off by setting the configuration parameter `no_alert_request` to true.

Implementation note: One may wonder when or how the behavior can make this automatic posting when the behavior is configured as a template. An instance is never spawned until an alert is received, but the alert parameters are posted by the behavior, creating a bit of a chicken or the egg conundrum. The alert request is actually posted by an instance of the behavior created ever so briefly at helm startup. At startup, the helm creates instances of *all* behaviors, even templates, to ensure the configuration parameters are correct. The ultimate confirmation of behavior parameter correctness is obtained by a behavior instance itself confirming each parameter. The helm will then immediately, before its first iteration, delete any behaviors temporarily created from template behaviors. During this brief startup period, the helm will invoke the function `onHelmStart()` for all behaviors. This function is defined at the IvPBehavior superclass level just like `onRunState()`. In the case of the collision avoidance behavior, this function is implemented to make the automatic alert configuration posting to `BCM_ALERT_REQUEST`.

1.5 Specifying the Priority Policy - the pwt_*_dist Parameters

The AvdColregs behavior may be configured to increase its priority as its range to the contact diminishes. The priority weight specified in its configuration represents the *maximum* possible priority applied to the behavior, presumably in close range to the contact. The range at which this maximum priority applies is specified in the pwt_inner_dist parameter. Likewise, the pwt_outer_dist parameter specifies a range to the contact where the priority weight becomes zero, regardless of the priority weight specified in the configuration file. This relationship is shown in Figure 1.

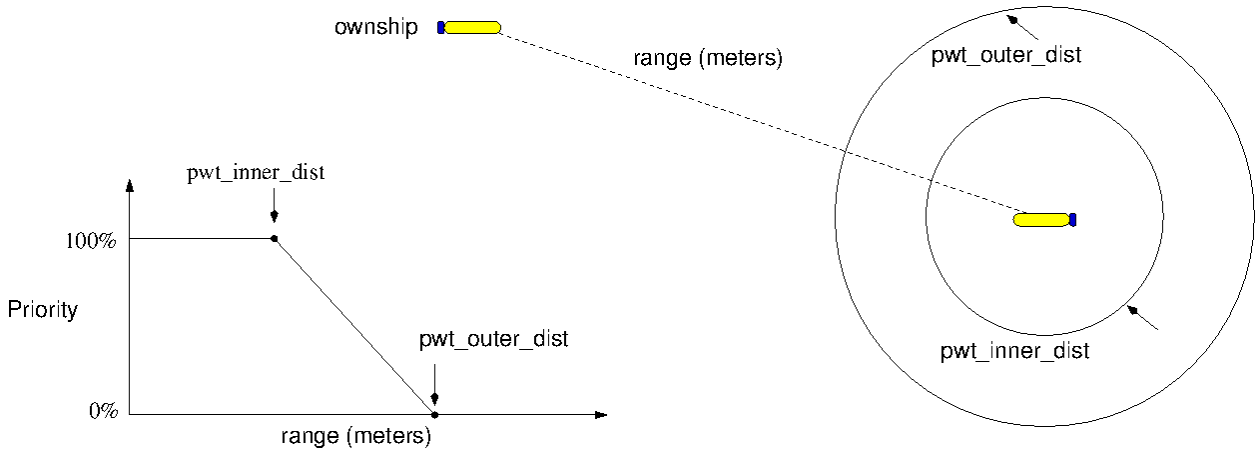

Figure 1: **Parameters for the BHV_AvdColregs behavior:** The *ownship* vehicle is the platform running the helm. The range between the two vehicles affects whether the behavior is active and with what priority weight. Beyond the range specified by pwt_outer_dist, the behavior is not be active. Within the range of pwt_inner_dist, the behavior is active with 100% of its configured priority weight.

By default, the priority weight decreases linearly between the two depicted ranges. The pwt_grade parameter allows the degradation from maximum priority to zero priority to fall more steeply by setting pwt_grade=quadratric.

1.6 Associating Utility to CPA of Candidate Maneuvers

- min_util_cpa_dist: The distance (in meters) between ownship and the contact at the closest point of approach (CPA) for a candidate maneuver, below which the behavior treats the distance as it would an actual collision between the two vehicles.
- max_util_cpa_dist: The distance (in meters) between ownship and the contact at the closest point of approach (CPA) for a candidate maneuver, above which the behavior treats the distance as having the maximum utility.

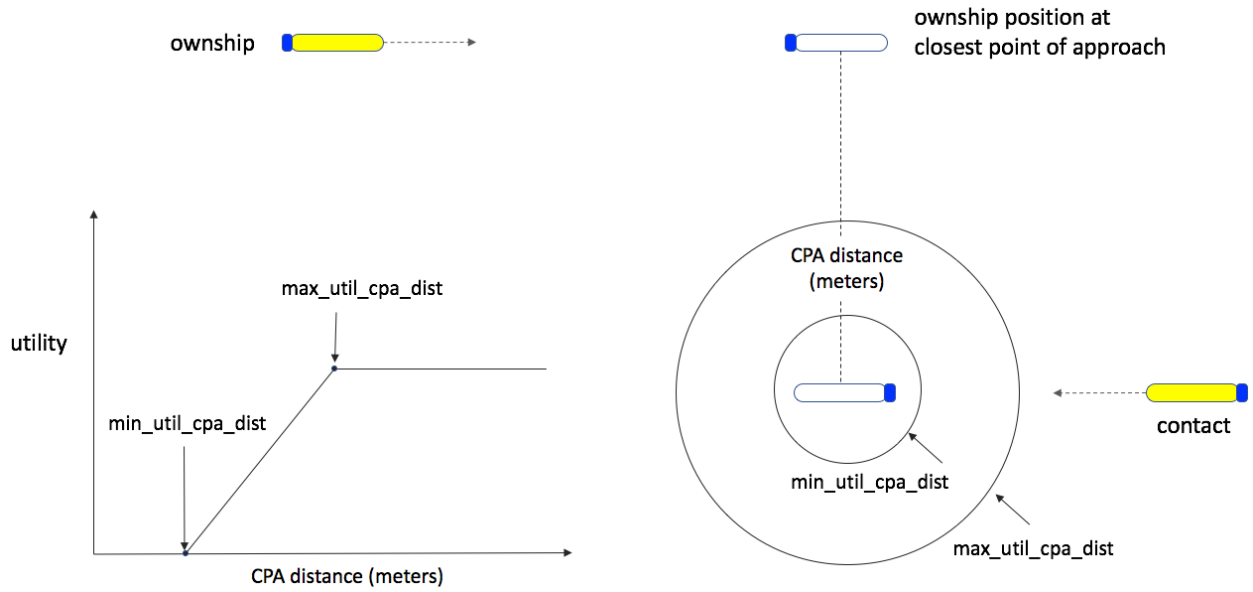

Figure 2: Parameters for the BHV_AvdColregs behavior. The *ownship* vehicle is the platform running the helm. The `collision_distance` is used when applying a utility metric to a calculated closest point of approach (CPA) for a candidate maneuver. A CPA less than or equal to the `collision_distance` is treated as an actual collision with the lowest utility rating.